

Lesson 7 — Advanced Prisma ORM

Node.js/Express

Game Plan

- Warm-Up
- Eager Loading (include)
- Aggregations & GroupBy
- Transactions
- Batch Operations
- Pagination & Filtering
- Raw SQL with `$queryRaw`
- Assignment Preview
- Wrap-Up

Warm-Up (5 min)

In chat or out loud:

1. What's something from Assignment 6 that gave you trouble?
2. If you needed to count how many tasks each user has, how would you do that in plain JavaScript vs SQL?

The N+1 Problem

```
// ✗ One query for users + one query per user for tasks
const users = await prisma.user.findMany();
for (const user of users) {
  const tasks = await prisma.task.findMany({ where: { userId: user.id } });
  user.tasks = tasks;
}
// 1 + N database queries
```

With 100 users, that's **101 queries**. That's the N+1 problem.

Eager Loading with `include`

```
// ✅ One query with a JOIN
const users = await prisma.user.findMany({
  include: {
    Task: true,
  },
});
```

Prisma generates a LEFT JOIN — one query for all users and their tasks.

Always prefer `include` over looping queries.

Filtering Inside `include`

```
const usersWithActiveTasks = await prisma.user.findMany({
  include: {
    Task: {
      where: { isCompleted: false },
      orderBy: { createdAt: "desc" },
      take: 5,
    },
  },
});
```

You can filter, sort, and limit the included relation data.

Aggregations

Count records in the database (not in JavaScript):

```
// Count total tasks for a user
const count = await prisma.task.count({
  where: { userId: 1 },
});

// Completion stats
const stats = await prisma.task.groupBy({
  by: ["isCompleted"],
  _count: { id: true },
  where: { userId: 1 },
});
```

`groupBy` is the Prisma equivalent of `GROUP BY` in SQL.

Transactions

When multiple operations must succeed or fail **together**:

```
const result = await prisma.$transaction(async (tx) => {  
  const user = await tx.user.create({  
    data: { email, name, hashedPassword },  
  });  
  await tx.task.createMany({  
    data: [  
      { title: "Complete your profile", userId: user.id },  
      { title: "Add your first task", userId: user.id },  
    ],  
  });  
  return user;  
});
```

If anything throws inside the transaction, everything rolls back.

Batch Operations

Create, update, or delete many records at once:

```
// Create many tasks
await prisma.task.createMany({
  data: [
    { title: "Task A", userId: 1 },
    { title: "Task B", userId: 1 },
  ],
  skipDuplicates: true,
});

// Mark old tasks complete
await prisma.task.updateMany({
  where: { createdAt: { lt: cutoffDate }, isCompleted: false },
  data: { isCompleted: true },
});
```

Much faster than looping with individual `create` / `update` calls.

Pagination

```
const page = parseInt(req.query.page) || 1;
const limit = parseInt(req.query.limit) || 10;
const skip = (page - 1) * limit;

const tasks = await prisma.task.findMany({
  where: { userId },
  skip,
  take: limit,
  orderBy: { createdAt: "desc" },
});

const total = await prisma.task.count({ where: { userId } });
```

Always return pagination metadata so the client knows how many pages exist.

Dynamic Filtering

Build a `where` clause based on query parameters:

```
const whereClause = { userId: global.user_id };

if (req.query.isCompleted !== undefined) {
  whereClause.isCompleted = req.query.isCompleted === "true";
}
if (req.query.find) {
  whereClause.title = { contains: req.query.find, mode: "insensitive" };
}
```

Each condition is added only when the query param is present.

Sorting with Query Params

```
const validFields = ["title", "createdAt", "isCompleted"];
const sortBy = req.query.sortBy || "createdAt";
const sortDir = req.query.sortDirection === "asc" ? "asc" : "desc";

const orderBy = validFields.includes(sortBy)
  ? { [sortBy]: sortDir }
  : { createdAt: "desc" };
```

Always validate `sortBy` against a whitelist to prevent errors.

When Prisma Isn't Enough

Prisma can't do:

- Complex JOINS
- Subqueries
- Advanced SQL functions

Use `$queryRaw` for those cases:

```
const stats = await prisma.$queryRaw`  
  SELECT u.name, COUNT(t.id) AS task_count  
  FROM users u  
  LEFT JOIN tasks t ON u.id = t.user_id  
  GROUP BY u.id, u.name  
  ORDER BY task_count DESC  
`;  
`;
```

Raw SQL Safety

Use template literals with `$queryRaw` — Prisma parameterizes them:

```
// ✅ SAFE — template literal
const results = await prisma.$queryRaw`
  SELECT * FROM tasks WHERE user_id = ${userId}
`;

// ❌ DANGEROUS — string concatenation
const results = await prisma.$queryRawUnsafe(
  `SELECT * FROM tasks WHERE user_id = ${userId}`
);
```

Same SQL injection rules apply to raw queries.

We Do — Design a Paginated Endpoint

Let's design `GET /api/tasks` with:

- Pagination (`page` , `limit`)
- Filter by `isCompleted`
- Sort by `createdAt` descending by default

What does the response shape look like?

```
{
  tasks: [...],
  pagination: { page, limit, total, pages, hasNext, hasPrev }
}
```

You Do (5 min)

Add a `min_date` filter to the task query:

- If `req.query.min_date` is present, only return tasks created on or after that date
- Combine it with any other active filters

```
if (req.query.min_date) {  
  // add to whereClause  
}
```

Assignment Preview

This week's assignment is labeled as long — budget your time.

You'll add to your app:

1. Eager loading on user/task queries (include relations)
2. Aggregation endpoints for analytics
3. A transaction for user registration (create user + welcome tasks)
4. Batch operations (createMany, updateMany, deleteMany)
5. Pagination + filtering + sorting on `GET /api/tasks`
6. A raw SQL search endpoint

Wrap-Up

In chat:

1. What's the N+1 problem and how does `include` fix it?
2. When would you use `createMany` over a loop of `create` calls?
3. Why must `$queryRaw` use template literals, not string concatenation?

Confidence Check

On a scale of 1–5:

How confident do you feel tackling this week's Prisma features?

Resources

- <https://www.prisma.io/docs/orm/prisma-client/queries/filtering-and-sorting>
- <https://www.prisma.io/docs/orm/prisma-client/queries/transactions>
- <https://www.prisma.io/docs/orm/prisma-client/queries/raw-database-access>
- Ask questions in Slack

Closing

This week:

Advanced Prisma — eager loading, aggregations, transactions, pagination.

Next week:

Authentication — JWT tokens, secure cookies, and protecting your app from real attacks.

See you then!